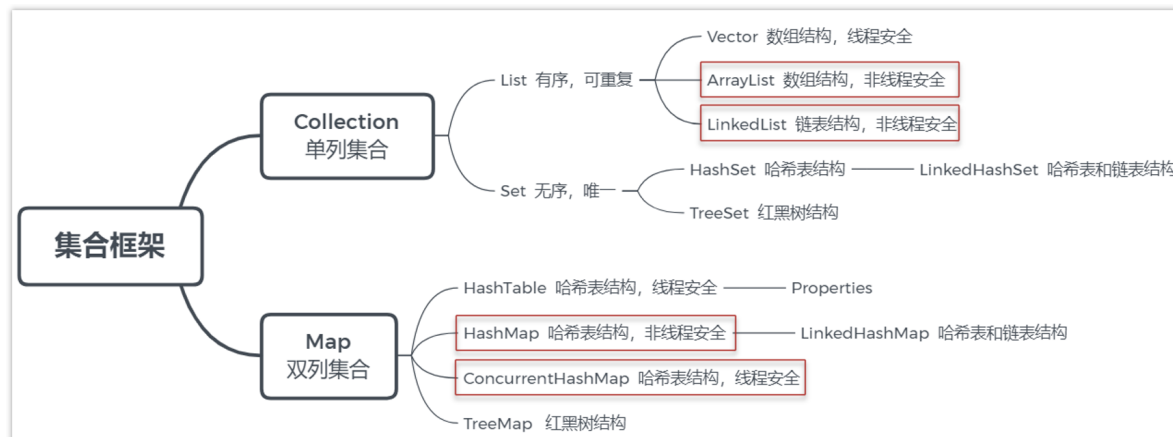


1.集合基础知识

1.Java集合有哪几种？

Java集合类主要由两个接口**Collection**和**Map**派生出来的，

一个是 **Collection** 接口，主要用于存放单一元素；另一个是 **Map** 接口，主要用于存放键值对。对于 **Collection** 接口，下面又有三个主要的子接口：**List**、**Set** 和 **Queue** (念q)。



哪些是线程安全哪些线程不安全？

java.util包下的集合类大部分都是线程不安全的，例如我们常用的HashSet、TreeSet、ArrayList、LinkedList、ArrayDeque、HashMap、TreeMap，这些都是线程不安全的集合类，但是它们的优点是

性能好。如果需要使用线程安全的集合类，则可以使用Collections工具类提供的synchronizedXxx()方法，将这些集合类包装成线程安全的集合类。

java.util包下也有线程安全的集合类，例如Vector、Hashtable。这些集合类都是比较古老的API，虽然

实现了线程安全，但是性能很差。所以即便是需要使用线程安全的集合类，也建议将线程不安全的集合类包装成线程安全集合类的方式，而不是直接使用这些古老的API。

从Java5开始，Java在java.util.concurrent包下提供了大量支持高效并发访问的集合类，它们既能包装良好的访问性能，有能包装线程安全。这些集合类可以分为两部分，它们的特征如下：

- 以Concurrent开头的集合类：
以Concurrent开头的集合类代表了支持并发访问的集合，它们可以支持多个线程并发写入访问，这些写入线程的所有操作都是线程安全的，但读取操作不必锁定。以Concurrent开头的集合类采用了更复杂的算法来保证永远不会锁住整个集合，因此在并发写入时有较好的性能。
- 以CopyOnWrite开头的集合类：
以CopyOnWrite开头的集合类采用复制底层数组的方式来实现写操作。当线程对此类集合执行读取操作时，线程将会直接读取集合本身，无须加锁与阻塞。当线程对此类集合执行写入操作时，集合会在底层复制一份新的数组，接下来对新的数组执行写入操作。由于对集合的写入操作都是对数组的副本执行操作，因此它是线程安全的。

2.集合的具体实现类

List

- ArrayList：Object[] 数组。
- Vector：Object[] 数组。

- `LinkedList`：双向链表(JDK1.6 之前为循环链表，JDK1.7 取消了循环)。

Map

- `HashMap`：JDK1.8 之前 `HashMap` 由数组+链表组成的，数组是 `HashMap` 的主体，链表则是主要为了解决哈希冲突而存在的（“拉链法”解决冲突）。JDK1.8 以后在解决哈希冲突时有了较大的变化，当链表长度大于阈值（默认为 8）（将链表转换成红黑树前会判断，如果当前数组的长度小于 64，那么会选择先进行数组扩容，而不是转换为红黑树）时，将链表转化为红黑树，以减少搜索时间。
- `LinkedHashMap`：`LinkedHashMap` 继承自 `HashMap`，所以它的底层仍然是基于拉链式散列结构即由数组和链表或红黑树组成。另外，`LinkedHashMap` 在上面结构的基础上，增加了一条双向链表，使得上面的结构可以保持键值对的插入顺序。同时通过对链表进行相应的操作，实现了访问顺序相关逻辑。
- `Hashtable`：数组+链表组成的，数组是 `Hashtable` 的主体，链表则是主要为了解决哈希冲突而存在的。
- `TreeMap`：红黑树（自平衡的排序二叉树）。

Set

- `HashSet` (无序，唯一)：基于 `HashMap` 实现的，底层采用 `HashMap` 来保存元素。
- `LinkedHashSet`：`LinkedHashSet` 是 `HashSet` 的子类，并且其内部是通过 `LinkedHashMap` 来实现的。
- `TreeSet` (有序，唯一)：红黑树(自平衡的排序二叉树)。

Queue

- `PriorityQueue`：`Object[]` 数组来实现小顶堆。
- `DelayQueue`：`PriorityQueue`。
- `ArrayDeque`：可扩容动态双向数组。

3.说说 List, Set, Queue, Map 四者的区别？

Java中的集合类主要由Collection和Map这两个接口派生而出，其中Collection接口又派生出三个子接口，分别是Set、List、Queue。所有的Java集合类，都是Set、List、Queue、Map这四个接口的实现类，这四个接口将集合分成了四大类，其中

- Set代表无序的，元素不可重复的集合；
- List代表有序的，元素可以重复的集合；
- Queue代表先进先出（FIFO）的队列；
- Map代表具有映射关系（key-value）的集合。

4.为什么要使用集合？

因为在实际开发中，存储的数据类型多种多样且数量不确定。这时，Java 集合就派上用场了。与数组相比，Java 集合提供了更灵活、更有效的方法来存储多个数据对象。Java 集合框架中的各种集合类和接口可以存储不同类型和数量的对象，同时还具有多样化的操作方式。相较于数组，Java 集合的优势在于它们的大小可变、支持泛型、具有内建算法等。总的来说，Java 集合提高了数据的存储和处理灵活性，可以更好地适应现代软件开发中多样化的数据需求，并支持高质量的代码编写。

5.什么是fail fast快速失败机制？

快速失败（fail-fast）是Java集合框架中的一种**错误检测机制**。

Fail-fast 机制主要用于确保集合在遍历过程中不被修改，从而保证数据的一致性和稳定性。

当多个线程对同一个ArrayList进行操作时，如果一个线程在遍历该集合的过程中，另一个线程同时尝试修改它（例如添加、删除元素），那么遍历线程会抛出 `ConcurrentModificationException` 异常。这种机制旨在防止并发修改导致的数据不一致问题。

6.什么是fail safe安全失败机制？

采用安全失败机制的集合容器，在遍历时不是直接在集合内容上访问的，而是先复制原有集合内容，在拷贝的集合上进行遍历。java.util.concurrent包下的容器都是安全失败，可以在多线程下并发使用，并发修改。

原理：由于迭代时是对原集合的拷贝进行遍历，所以在遍历过程中对原集合所作的修改并不能被迭代器检测到，所以不会触发Concurrent Modification Exception。

缺点：基于拷贝内容的优点是避免了Concurrent Modification Exception，但同样地，迭代器并不能访问到修改后的内容，即：迭代器遍历的是开始遍历那一刻拿到的集合拷贝，在遍历期间原集合发生的修改迭代器是不知道的。

7.如何让一个集合不能被修改？

可以采用Collections包下的unmodifiableMap/unmodifiableList/unmodifiableSet方法，通过这个方法返回的集合，是不可以修改的。如果修改的话，会抛出 java.lang.UnsupportedOperationException异常。

```
List<String> list = new ArrayList<>();
list.add("x");
Collection<String> clist = Collections.unmodifiableCollection(list);
clist.add("y"); // 运行时此行报错
System.out.println(list.size());
```

对于List/Set/Map集合，Collections包都有相应的支持。

那使用final关键字进行修饰可以实现吗？

答案是不可以。

final关键字修饰的成员变量如果是引用类型的话，则表示这个引用的地址值是不能改变的，但是这个引用所指向的对象里面的内容还是可以改变的。

而集合类都是引用类型，用final修饰的话，集合里面的内容还是可以修改的。

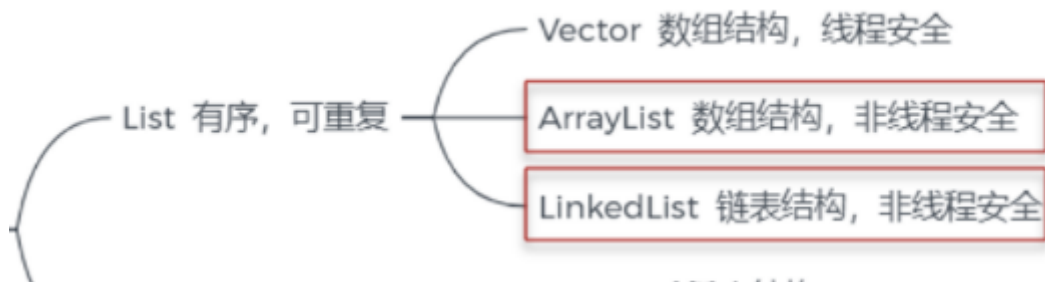
引用类型有哪些？

在Java中，引用类型主要包括**类（Class）、接口（Interface）、数组（Array）以及基于这些结构的其他数据类型如字符串（String）和枚举类型（Enum）**。

Java将数据类型主要分为两大类：基本数据类型和引用数据类型。基本数据类型包括byte、short、int、long、float、double、char和boolean，它们直接存储值而非对象的引用。引用类型的变量则存储的是对象在内存中的地址，即引用了对象的位置。

2.List

-1.List有哪些类？



0.什么是ArrayList?

`ArrayList` 的底层是动态数组，它的容量能动态增长。在添加大量元素前，应用可以使用 `ensureCapacity` 操作增加 `ArrayList` 实例的容量。`ArrayList` 继承了 `AbstractList`，并实现了 `List` 接口

0.有哪些线程安全的List?

参考答案

- **Vector**
Vector是比较古老的API，虽然保证了线程安全，但是由于效率低一般不建议使用。
- **Collections.SynchronizedList**
SynchronizedList是Collections的内部类，Collections提供了synchronizedList方法，可以将一个线程不安全的List包装成线程安全的List，即SynchronizedList。它比Vector有更好的扩展性和兼容性，但是它所有的方法都带有同步锁，也不是性能最优的List。
- **CopyOnWriteArrayList**
CopyOnWriteArrayList是Java 1.5在java.util.concurrent包下增加的类，它采用复制底层数组的方式来实现写操作。当线程对此类集合执行读取操作时，线程将会直接读取集合本身，无须加锁与阻塞。当线程对此类集合执行写入操作时，集合会在底层复制一份新的数组，接下来对新的数组执行写入操作。由于对集合的写入操作都是对数组的副本执行操作，因此它是线程安全的。在所有线程安全的List中，它是性能最优的方案。

1.ArrayList 和 Array（数组）的区别?

`ArrayList` 内部基于动态数组实现，比 `Array`（静态数组）使用起来更加灵活：

- `ArrayList` 会根据实际存储的元素动态地扩容或缩容，而 `Array` 被创建之后就不能改变它的长度了。
- `ArrayList` 允许你使用泛型来确保类型安全，`Array` 则不可以。
- `ArrayList` 中只能存储对象。对于基本类型数据，需要使用其对应的包装类（如 `Integer`、`Double` 等）。`Array` 可以直接存储基本类型数据，也可以存储对象。
- `ArrayList` 支持插入、删除、遍历等常见操作，并且提供了丰富的API操作方法，比如 `add()`、`remove()` 等。`Array` 只是一个固定长度的数组，只能按照下标访问其中的元素，不具备动态添加、删除元素的能力。
- `ArrayList` 创建时不需要指定大小，而 `Array` 创建时必须指定大小

2.ArrayList 和 Vector 的区别?

- `ArrayList` 是 `List` 的主要实现类，底层使用 `Object[]` 存储，适用于频繁的查找工作，线程不安全。
- `Vector` 是 `List` 的古老实现类，底层使用 `Object[]` 存储，线程安全。
- `ArrayList`在内存不够时扩容为原来的1.5倍，`Vector`是扩容为原来的2倍。

3.ArrayList 可以添加 null 值吗?

`ArrayList` 中可以存储任何类型的对象，包括 `null` 值。

4.ArrayList 与 LinkedList的区别

- ArrayList的实现是基于数组，LinkedList的实现是基于双向链表；
- 对于随机访问ArrayList要优于LinkedList，ArrayList可以根据下标以 $O(1)$ 时间复杂度对元素进行随机访问，而LinkedList的每一个元素都依靠地址指针和它后一个元素连接在一起，查找某个元素的时间复杂度是 $O(N)$ ；
- 对于插入和删除操作，LinkedList要优于ArrayList，因为当元素被添加到LinkedList任意位置的时候，不需要像ArrayList那样重新计算大小或者是更新索引；
- LinkedList比ArrayList更占内存，因为LinkedList的节点除了存储数据，还存储了两个引用，一个指向前一个元素，一个指向后一个元素

5.ArrayList扩容原理

ArrayList有三种构造方法，**无参构造方法将创建一个空的ArrayList，其内部使用一个默认容量为10的空数组初始化。如果通过指定初始容量来构造ArrayList，那么会创建一个具有该初始容量的数组。**第三种构造方法允许传入一个集合，并将其所有元素添加到ArrayList中。

- 无参构造方法扩容过程如下

ArrayList的底层是动态数组，默认第一次插入元素时创建大小为10的数组。当调用add方法添加一个元素时，首先会确保当前ArrayList维护的数组具有存储新元素的能力。如果数组的容量不足以存储新元素，那么就会通过grow方法进行扩容。扩容的方式是将数组的容量扩大到原来的1.5倍，然后将原数组的数据复制到新的数组中。最后，将新元素添加到数组的末尾

6. 面试题-ArrayList list=new ArrayList(10)中的list扩容几次

在ArrayList的源码中提供了一个带参数的构造方法，这个参数就是指定的集合初始长度，所以给了一个10的参数，就是指定了集合的初始长度是10，这里面并没有扩容。

7.谈谈CopyOnWriteArrayList的原理

CopyOnWriteArrayList是Java并发包里提供的并发类，简单来说它就是一个**线程安全且读操作无锁的ArrayList**。CopyOnWriteArrayList允许线程并发访问读操作，这个时候是没有加锁限制的，性能较高。正如其名字一样，**在写操作时会复制一份新的List，在新的List上完成写操作，然后再将原引用指向新的List。这样就保证了写操作的线程安全。**

- 优点：读操作性能很高，因为无需任何同步措施，比较适用于读多写少的并发场景。在遍历传统的List时，若中途有别的线程对其进行修改，则会抛出ConcurrentModificationException异常。而CopyOnWriteArrayList由于其“读写分离”的思想，遍历和修改操作分别作用在不同的List容器，所以在使用迭代器进行遍历时候，也就不会抛出ConcurrentModificationException异常了。
- 缺点：一是内存占用问题，毕竟每次执行写操作都要将原容器拷贝一份，数据量大时，对内存压力较大，可能会引起频繁GC。二是无法保证实时性，Vector对于读写操作均加锁同步，可以保证读和写的强一致性。而CopyOnWriteArrayList由于其实现策略的原因，写和读分别作用在新老不同容器上，在写操作执行过程中，读不会阻塞但读取到的却是老容器的数据。

7.LinkedList 为什么不能实现 RandomAccess 接口？

`RandomAccess` 是一个标记接口，用来表明实现该接口的类支持随机访问（即可以通过索引快速访问元素）。由于 `LinkedList` 底层数据结构是链表，内存地址不连续，只能通过指针来定位，不支持随机快速访问，所以不能实现 `RandomAccess` 接口

8.怎么在遍历 ArrayList 时移除一个元素?

foreach删除会导致快速失败问题，可以使用迭代器的 remove() 方法。

```
Iterator itr = list.iterator();
while(itr.hasNext()) {
    if(itr.next().equals("jay") {
        itr.remove();
    }
}
```

9.什么是ArrayList的快速失败fail fast机制

ArrayList的快速失败（fail-fast）是Java集合框架中的一种错误检测机制。

当多个线程对同一个ArrayList进行操作时，如果一个线程在遍历该集合的过程中，另一个线程同时尝试修改它（例如添加、删除元素），那么遍历线程会抛出 `ConcurrentModificationException` 异常。

Fail-fast 机制主要用于确保集合在遍历过程中不被修改，从而保证数据的一致性和稳定性。

10.如何实现数组和List之间的转换

数组转list，可以使用jdk自动的一个工具类Arrays，里面有一个asList方法可以转换为数组

List 转数组，可以直接调用list中的toArray方法、，需要给一个参数，指定数组的类型，需要指定数组的长度。

```
//数组转List
public static void testArray2List(){
    String[] strs = {"aaa","bbb","ccc"};
    List<String> list = Arrays.asList(strs);
    for (String s : list) {
        System.out.println(s);
    }
}

//List转数组
public static void testList2Array(){
    List<String> list = new ArrayList<String>();
    list.add("aaa");
    list.add("bbb");
    list.add("ccc");
    String[] array = list.toArray(new String[list.size()]);
    for (String s : array) {
        System.out.println(s);
    }
}
```

11.用Arrays.asList转List后，如果修改了数组内容，list受影响吗？List用toArray转数组后，如果修改了List内容，数组受影响吗

Arrays.asList转换list之后，如果修改了数组的内容，list会受影响，因为它的底层使用的Arrays类中的一个内部类ArrayList来构造的集合，在这个集合的构造器中，把我们传入的这个集合进行了包装而已，**最终指向的都是同一个内存地址**

list用了toArray转数组后，如果修改了list内容，数组不会受影响，**当调用了toArray以后，在底层它是进行了数组的拷贝，跟原来的元素就没啥关系了，所以即使list修改了以后，数组也不受影响。**

12.ArrayList 和 LinkedList 不是线程安全的，你们在项目中是如何解决这个的线程安全问题的？

第一：我们使用这个集合，优先在方法内使用，定义为局部变量，这样的话，就不会出现线程安全问题。

第二：如果非要在成员变量中使用的话，可以使用线程安全的集合来替代

ArrayList可以用CopyOnWriteArrayList

LinkedList 换成ConcurrentLinkedQueue来使用

3.Map相关面试题

0.Map有哪些类？



Map接口有很多实现类，其中比较常用的有HashMap、LinkedHashMap、TreeMap、ConcurrentHashMap。

对于不需要排序的场景，优先考虑使用HashMap，因为它是性能最好的Map实现。如果需要保证线程安全，则可以使用ConcurrentHashMap。它的性能好于Hashtable，因为它在put时采用分段锁/CAS的加锁机制，而不是像Hashtable那样，无论是put还是get都做同步处理。

对于需要排序的场景，如果需要按插入顺序排序则可以使用LinkedHashMap，如果需要将key按自然顺序排列甚至是自定义顺序排列，则可以选择TreeMap。如果需要保证线程安全，则可以使用Collections工具类将上述实现类包装成线程安全的Map。

TreeMap 基于红黑树实现。

HashMap 1.7基于哈希表实现，1.8基于数组+链表+红黑树。

HashTable 和 **HashMap** 类似，但它是线程安全的，这意味着同一时刻多个线程可以同时写入 **HashTable** 并且不会导致数据不一致。它是遗留类，不应该去使用它。现在可以使用 **ConcurrentHashMap** 来支持线程安全，并且 **ConcurrentHashMap** 的效率会更高(1.7 **ConcurrentHashMap** 引入了分段锁, 1.8 引入了红黑树)。

LinkedHashMap 使用双向链表来维护元素的顺序，顺序为插入顺序或者最近最少使用(LRU)顺序。

1.HashMap 和 Hashtable 的区别

- 线程是否安全：** **HashMap** 是非线程安全的，**Hashtable** 是线程安全的,因为 **Hashtable** 内部的方法基本都经过 **synchronized** 修饰。（如果你要保证线程安全的话就使用 **ConcurrentHashMap**

吧！)；

- **效率：**因为线程安全的问题，`HashMap` 要比 `Hashtable` 效率高一点。另外，`Hashtable` 基本被淘汰，不要在代码中使用它；
- **对 Null key 和 Null value 的支持：**`HashMap` 可以存储 null 的 key 和 value，但 null 作为键只能有一个，null 作为值可以有多个；`Hashtable` 不允许有 null 键和 null 值，否则会抛出 `NullPointerException`。
- **初始容量大小和每次扩充容量大小的不同：**① 创建时如果不指定容量初始值，`Hashtable` 默认的初始大小为 11，之后每次扩充，容量变为原来的 $2n+1$ 。`HashMap` 默认的初始化大小为 16。之后每次扩充，容量变为原来的 2 倍。② 创建时如果给定了容量初始值，那么 `Hashtable` 会直接使用你给定的大小，而 `HashMap` 会将其扩充为 2 的幂次方大小（`HashMap` 中的 `tableSizeFor()` 方法保证，下面给出了源代码）。也就是说 `HashMap` 总是使用 2 的幂作为哈希表的大小，后面会介绍到为什么是 2 的幂次方。
- **底层数据结构：**JDK1.8 以后的 `HashMap` 在解决哈希冲突时有了较大的变化，当链表长度大于阈值（默认为 8）时，将链表转化为红黑树（将链表转换成红黑树前会判断，如果当前数组的长度小于 64，那么会选择先进行数组扩容，而不是转换为红黑树），以减少搜索时间（后文中我会结合源码对这一过程进行分析）。`Hashtable` 没有这样的机制。

2. HashSet与HashMap的区别

(1) `HashSet` 实现了 `Set` 接口，仅存储对象；`HashMap` 实现了 `Map` 接口，存储的是键值对。

(2) `HashSet` 底层其实是用 `HashMap` 实现存储的，`HashSet` 封装了一系列 `HashMap` 的方法。依靠 `HashMap` 来存储元素值，（利用 `HashMap` 的 key 键进行存储），而 value 值默认为 `Object` 对象。所以 `HashSet` 也不允许出现重复值，判断标准和 `HashMap` 判断标准相同，两个元素的 `hashCode` 相等并且通过 `equals()` 方法返回 `true`。

4. HashMap的底层实现(jdk1.7和jdk1.8有区别)

• jdk1.7

JDK7中的HashMap，是基于数组+链表来实现的，它的底层维护一个 `Entry` 数组。它会根据计算的 `hashCode` 将对应的 KV 键值对存储到该数组中，一旦发生 `hashCode` 冲突，那么就会将该 KV 键值对放到对应的已有元素的后面，此时便形成了一个链表式的存储结构。

JDK7 中 `HashMap` 的实现方案有一个明显的缺点，即当 Hash 冲突严重时，在桶上形成的链表会变得越长，这样在查询时的效率就会越来越低，其时间复杂度为 $O(N)$ 。

• jak1.8

JDK8中的HashMap，是基于数组+链表+红黑树来实现的，它的底层维护一个 `Node` 数组。当链表长度大于阈值（默认为 8）外加数组长度大于 64 时时，将链表转化为红黑树，以减少搜索时间。**这么做主要是在查询的时间复杂度上进行优化**，链表为 $O(N)$ ，而红黑树一直是 $O(\log N)$ ，可以大大的提高查找性能。

1.线性探测法和链地址法

线性探测法是开放地址法的一种，用于处理哈希表中的冲突问题。

线性探测法的核心在于解决哈希冲突时不采用链表，而是通过探测散列表中的下一个位置来寻找空位。具体来说，如果一个关键字通过散列函数计算的地址已经被占用，它会尝试下一个地址，即当前地址加一（也可以考虑为负数或固定步长，视情况而定），直到找到一个空位为止。这种方法在实际操作中简单且易于实现，但当哈希表较为拥挤时，可能会导致很多空闲位置之间出现很多“堆积”，增加了平均查找时间。

链地址法则是将具有相同哈希值的所有元素链接在同一个链表中。

链地址法也被称为拉链法，它采用了不同的策略来解决哈希冲突。在这个方法中，每个哈希到同一个值的元素都会被插入到对应哈希值下的链表中。这意味着如果多个元素的哈希值相同，它们会被放在同一个链表里，以此来区分这些具有相同哈希值的元素。**链地址法的优点是可以减少在插入和查找过程中的平均比较次数，因为每次比较的都是同义词节点。然而，这种方法需要额外的空间来存储指针信息。**

总之，这两种方法各有利弊，在不同的场景下可能会选择不同的方法来优化性能。线性探测法更适用于哈希冲突较少时使用，而链地址法适用于处理大量冲突的情况。在实际应用中，选择哪种方法取决于具体的需求和场景。

2.红黑树

1.红黑树的介绍

(1) 概述

红黑树 (Red Black Tree)：也是一种自平衡的二叉搜索树(BST)，之前叫做平衡二叉B树 (Symmetric Binary B-Tree)

(2) 红黑树的特质

性质1：节点要么是**红色**,要么是**黑色**

性质2：根节点是**黑色**

性质3：叶子节点都是黑色的空节点

性质4：红黑树中红色节点的子节点都是黑色

性质5：从任一节点到叶子节点的所有路径都包含相同数目的黑色节点

在添加或删除节点的时候，如果不符合这些性质会发生旋转，以达到所有的性质，保证红黑树的平衡

(3) 红黑树的复杂度

- 查找：
 - 红黑树也是一棵BST（二叉搜索树）树，查找操作的时间复杂度为： $O(\log n)$
- 添加：
 - 添加先要从根节点开始找到元素添加的位置，时间复杂度 $O(\log n)$
 - 添加完成后涉及到复杂度为 $O(1)$ 的旋转调整操作
 - 故整体复杂度为： $O(\log n)$
- 删除：
 - 首先从根节点开始找到被删除元素的位置，时间复杂度 $O(\log n)$
 - 删除完成后涉及到复杂度为 $O(1)$ 的旋转调整操作
 - 故整体复杂度为： $O(\log n)$

2.为什么选红黑树，和二叉搜索树、AVL树（平衡二叉树）有什么区别？

- **二叉搜索树**：在二叉搜索树中，左子节点的值小于根节点的值，右子节点的值大于根节点的值。这使得二叉搜索树在查找操作上具有优势。然而，**二叉搜索树可能退化为线性结构，即链表**，当数据插入顺序有序或接近有序时，其查找效率会大大降低，时间复杂度可能达到 $O(n)$ 。
- **AVL树**：是一种高度平衡的二叉搜索树，它要求每个节点的左右子树的高度差不超过1。这种严格的平衡条件使得AVL树在查找操作上具有很高的效率，时间复杂度为 $O(\log n)$ 。然而，**为了维护这种严格的平衡，AVL树在插入和删除操作时需要进行频繁的旋转调整，这增加了维护成本。因此，AVL树适用于查找操作频繁但插入和删除操作较少的场景。**
- **红黑树**：红黑树是一种近似平衡的二叉搜索树，它通过一系列性质（如节点颜色、黑高）来维护树的平衡。与AVL树相比，红黑树的平衡条件相对宽松，因此在插入和删除操作时的维护成本较低。

虽然红黑树的查找效率略低于AVL树，但其综合性能较好，适用于各种操作（插入、删除和查找）都较频繁的场景。此外，红黑树的高度近似为 $2\log n$ ，在实际应用中表现出良好的性能。

3.在解决 hash 冲突的时候，为什么选择先用链表，再转红黑树？

因为红黑树需要进行左旋，右旋，变色这些操作来保持平衡，而单链表不需要，所以元素的插入操作非常高效。所以，当元素个数小于8个的时候，采用链表结构可以保证查询性能。而当元素个数大于8个的时候并且数组容量大于等于64，会采用红黑树结构。因为红黑树搜索时间复杂度是 $O(\log n)$ ，而链表是 $O(n)$ ，在n比较大的时候，使用红黑树可以加快查询速度。

4.为什么链表改为红黑树的阈值是 8？

理想情况下使用随机的哈希码，容器中节点分布在 hash 桶中的频率遵循泊松分布，按照泊松分布的计算公式计算出了桶中元素个数和概率的对照表，可以看到链表中元素个数为 8 时的概率已经非常小，再多的就更少了，所以原作者在选择链表元素个数时选择了 8，是根据概率统计而选择的。

5.红黑树会退化为 $O(n)$ 的查找时间复杂度吗

红黑树理论上不会退化为 $O(n)$ 的查找时间复杂度。红黑树是一种自平衡二叉查找树，它通过对树中的节点进行着色和旋转维护了特定的性质，从而确保了其查找、插入和删除操作的时间复杂度在最坏情况下仍为 $O(\log n)$ 。

以下是确保红黑树效率的关键要素：

1. **着色规则**：红黑树中的每一个节点要么是红色，要么是黑色。其中，红色的节点不能相邻，这意味着没有两个红色节点可以成为对方的父子关系。
2. **树的平衡**：通过旋转操作来保持树的平衡。如果添加或删除节点后破坏了红黑树的性质，可以通过左旋或右旋来调整节点的位置，重新满足红黑树的性质。
3. **黑高度相等**：从根节点到任何叶子节点的路径上，黑色节点的数量都相等，这个性质保证了最长路径不会超过最短路径的两倍长度。
4. **节点插入策略**：新插入的节点总是作为红色节点，并遵循一定的规则来保证插入后仍是一棵有效的红黑树，必要时会通过旋转和重新着色来维护树的结构。
5. **修复机制**：当执行删除操作时，可能会有一些红黑树性质被破坏，但算法提供了修复机制来重新调整树的结构，以恢复其有效性。

然而，尽管红黑树设计得旨在防止性能退化，但在极端情况下（例如，频繁的序列性插入和删除导致多次调整后仍出现不平衡），如果没有适当的旋转和重平衡，极端不平衡的二叉查找树可能会接近链表的性能。这种情况在实际中很少见，因为红黑树的操作本身就是为了防止这种极端情况的发生。

总的来说，红黑树是设计用来在各种操作下维持对数级性能的，即使在最坏的情况下也保证了 $O(\log n)$ 的时间复杂度，不会出现 $O(n)$ 的查找时间复杂度。

5.HashMap读写时间复杂度是多少？

HashMap的读取（get）和写入（put）操作的平均时间复杂度通常为 $O(1)$ 。

在理想情况下，即哈希函数分布均匀且没有冲突时，HashMap的读取和写入操作可以直接定位到数组的相应位置，因此时间复杂度为 $O(1)$ 。但在最坏的情况下，如果出现大量的哈希冲突，这些操作的时间复杂度可能会退化为 $O(n)$ ，其中n是HashMap中元素的数量。

6.HashMap 的长度为什么是 2 的幂次方

key的Hash 值是int型(32位)，范围值-很大，只要哈希函数映射得比较均匀松散，一般应用是很难出现碰撞的。但问题是一个 40 亿长度的数组，内存是放不下的。所以这个散列值是不能直接拿来用的。用之前还要先做对数组的长度取模运算，得到的余数才能用来要存放的位置也就是对应的数组下标。

如果数组大小是2的n次方的化，我们也可以用hash值位与运算&(数组大小-1)，这样和取模效果一样，而且二进制的位运算比取模效率高得多。

7.解决hash冲突的办法有哪些？HashMap用的哪种？（链地址法）

解决Hash冲突方法有：开放定址法、再哈希法、链地址法。HashMap中采用的是链地址法。

- 开放定址法基本思想就是，如果 $p=H(\text{key})$ 出现冲突时，则以 p 为基础，再次hash， $p_1=H(p)$ ，如果 p_1 再次出现冲突，则以 p_1 为基础，以此类推，直到找到一个不冲突的哈希地址 p_i 。因此开放定址法所需要的hash表的长度要大于等于所需要存放的元素，而且因为存在再次hash，所以只能在删除的节点上做标记，而不能真正删除节点。
- 再哈希法提供多个不同的hash函数，当 $R_1=H_1(\text{key}_1)$ 发生冲突时，再计算 $R_2=H_2(\text{key}_1)$ ，直到没有冲突为止。这样做虽然不易产生堆集，但增加了计算的时间。
- 链地址法：将链表和数组相结合。也就是说创建一个链表数组，数组中每一格就是一个链表。若遇到哈希冲突，则将冲突的值加到链表中即可。

8.HashMap的put方法的具体流程

1. 判断键值对数组table是否为空或为null，若位空第一次扩容(resize:默认情况下，如果没有指定初始容量，则扩容的大小为16)
2. 通过hash算法根据键值key计算hash值得到数组索引
3. 判断索引处有没有存在元素，没有就直接插入
4. 如果存在元素且该key已存在，则直接覆盖其value；
5. 如果索引处存在元素且该key不存在，则遍历插入，有两种情况，一种是链表形式就直接遍历到尾端插入，一种是红黑树就按照红黑树结构插入
6. 链表的数量大于阈值8且数组长度大于64，就要转换成红黑树的结构
7. 插入成功后，判断实际存在的键值对数量size是否超多了最大容量threshold（数组长度*0.75），如果超过，进行扩容。

9.HashMap的get方法的具体流程

HashMap的get方法的具体流程涉及查找哈希表以定位键值对。具体步骤如下：

1. **计算哈希值**：首先，通过传入的 key 调用其 hashCode() 方法获取哈希值。这个值将用于确定键值对在内部数组中的位置。
2. **定位数组索引**：使用哈希值与数组长度减一（length - 1）进行位与运算（&），得到数组中对应的索引位置。
3. **检查节点**：在得到的索引位置上，首先检查是否有节点存在。如果该位置为空，则直接返回 null，表示没有找到对应的 value。
4. **比较键值**：如果有节点存在，接着比较节点的 key 是否与传入的 key 相等。如果相等，则返回该节点的 value。
5. **处理链表或红黑树**：如果节点的 key 不相等，且该索引位置是一个链表或红黑树（当链表长度超过一定阈值时会转换为红黑树以提高搜索效率），则需要遍历这个数据结构，继续比较 key 直到找到匹配的键值对或者遍历结束。
6. **返回结果**：如果在链表或红黑树中找到了对应的 key，则返回其 value；如果没有找到，则返回 null。

综上所述，HashMap的 get 方法是通过计算哈希值和比较键值来快速定位并返回对应 value 的过程。

9.HashMap的扩容机制

- 在添加元素或初始化的时候需要调用resize方法进行扩容，第一次添加数据初始化数组长度为16，以后每次扩容都是达到了**扩容阈值 (当前数组长度 * 0.75)**
- 每次扩容的时候，都是扩容之前容量的2倍；
- 扩容之后，会新创建一个数组，需要把老数组中的数据挪动到新的数组中
- 没有hash冲突节点，则直接使用 $e.hash \& (newCap - 1)$ (值等于 $e.hash \% newCap$)计算新数组的索引位置
- 如果是红黑树，走红黑树的添加
- 如果是链表，则需要遍历链表，可能需要拆分链表，判断 $(e.hash \& oldCap)$ 是否为0，若为0该元素的位置停留在原始位置，否则移动到原始位置+增加的数组大小这个位置上

0.为什么容量是以2的次方扩充的？

是为了提高性能使用足够大的数组，二是为了能使用位运算代替取模运算(据说提升了5~8倍)。

1.为什么要判断 $e.hash \& oldCap == 0$

举例:原数组容量为16，存有3个元素，他们hash值分别为5, 21, 37，他们都存在下标为5的链表里，扩容后数组容量为32，他们对应的新数组下标分别为5, 21, 5，可用发现，hash值为5和37的都满足 $e.hash \& oldCap == 0$ ，因此他们下标不变；而hash为21的不满足 $e.hash \& oldCap == 0$ ，此时他的下标变为原下标+oldCap。这样可以省去重新计算hash值的时间

2.100个数据放入hashmap要扩容到多少

256；应为扩容到128时 $128 * 0.75 < 100$ ，所以要再扩容一次

10.HashMap的散列/寻址算法

这个哈希方法首先通过key的hashCode()方法计算出key的hashCode值，然后通过这个hashCode值右移16位后的二进制与自己本身进行按位**异或运算**得到最后的hash值。

```
int idx = (len-1) & (hash ^ (hash>>>16))
```

1.JDK 8 为什么要 hashCode 异或其右移十六位的值？

因为在JDK 7 中扰动了 4 次，计算 hash 值的性能会稍差一点点。

从速度、功效、质量来考虑，JDK 8 优化了高位运算的算法，通过hashCode()的高16位异或低16位实现：`(h = k.hashCode()) ^ (h >>> 16)`。

这么做可以在数组 table 的 length 比较小的时候，也能保证考虑到高低Bit都参与到Hash的计算中，同时不会有太大的开销。

2.为什么 hash 值要与length-1相与？

- 把 hash 值对数组长度取模运算，模运算的消耗很大，没有位运算快。
- 当 length 总是 2 的n次方时， $h \& (length-1)$ 运算等价于对length取模，也就是 $h \% length$ ，但是位与运算比 取模运算 具有更高的效率。

3.hashCode()方法是啥？

一个Object类的方法，将与对象相关的信息映射成一个哈希值，默认的实现hashCode值是根据内存地址换算出来。

11.HashMap为什么线程不安全？

- **扩容造成死循环**：在jdk1.7版本的HashMap中，扩容操作可能会引发链表形成环状结构，导致无限循环的问题。具体来说，**当两个线程同时对一个链表进行put操作，并且触发了扩容，那么在转移元素到新数组的过程中，由于头插法的使用，链表元素顺序会被反转。如果两个线程交替执行，就可能出现环形链表的情况，进而在遍历或操作该链表时进入死循环。**

- 在JDK1.7及之前版本的HashMap中，链表节点的插入采用头插法。当有多个线程同时对链表进行操作时，可能会发生以下情况：

1. 线程A和线程B同时访问一个桶位，发现需要进行扩容操作。
2. 线程A获取到锁，开始进行扩容操作，将链表中的节点重新定位到新的桶位。
3. 线程B等待线程A释放锁，然后获取到锁，也开始进行扩容操作。
4. 线程A在重新定位节点时，可能会将节点插入到链表头部，这时线程B可能已经将部分节点重新定位到了新的桶位。
5. 由于头插法的特性，线程A在插入节点时，可能会将已经重新定位的节点再次插入到链表头部，导致链表中的节点指向错误的位置。
6. 这样，链表中的节点形成了一个环形结构，使得查询元素的操作陷入死循环无法结束。

因此，头插法在多线程环境下容易导致链表中的节点指向错误的位置，从而形成环形链表。为了避免这个问题，从JDK1.8开始，HashMap的实现采用了尾插法，并且在扩容操作时使用了分段锁技术，提高了多线程环境下的性能和稳定性。

- **扩容造成数据丢失**：除了死循环问题外，jdk1.7版本在多线程环境下扩容还可能导致数据丢失。在多线程同时进行put操作和扩容操作的情况下，某些元素可能因为并发操作的错误而被覆盖或者遗漏。
- **数据覆盖**：当两个线程计算得出相同的哈希值并且要插入相同的位置时，后来的操作可能会覆盖先前线程的插入结果，导致数据一致性问题。
 - 数据覆盖举个例子：
 - 两个线程 1,2 同时进行 put 操作，并且发生了哈希冲突（hash 函数计算出的插入下标是相同的）。
 - 不同的线程可能在不同的时间片获得 CPU 执行的机会，当前线程 1 执行完哈希冲突判断后，由于时间片耗尽挂起。线程 2 先完成了插入操作。
 - 随后，线程 1 获得时间片，由于之前已经进行过 hash 碰撞的判断，所有此时会直接进行插入，这就导致线程 2 插入的数据被线程 1 覆盖了。

综上所述，HashMap的线程不安全主要体现在jdk1.7中的死循环和数据丢失问题以及jdk1.8中的数据覆盖问题上。这些问题的根本原因在于HashMap的设计并未考虑多线程并发操作的情况，导致多个线程之间的操作互相干扰，从而产生错误

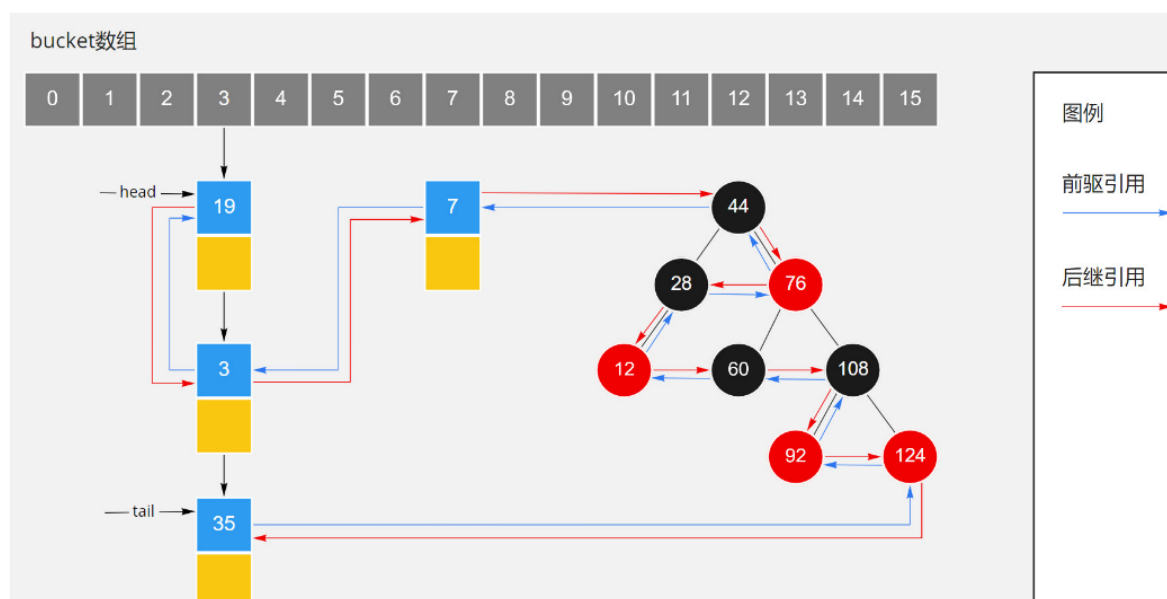
12. LinkedHashMap底层原理？

LinkedHashMap使用双向链表来维护key-value对的顺序（其实只需要考虑key的顺序），该链表负责

维护Map的迭代顺序，迭代顺序与key-value对的插入顺序保持一致。

LinkedHashMap可以避免对HashMap、Hashtable里的key-value对进行排序（只要插入key-value对时保持顺序即可），同时又可避免使用TreeMap所增加的成本。

LinkedHashMap需要维护元素的插入顺序，因此性能略低于HashMap的性能。但因为它以链表来维护内部顺序，所以在迭代访问Map里的全部元素时将有较好的性能。



13.HashMap与ConcurrentHashMap有什么区别？

HashMap和ConcurrentHashMap都是Java中常用的哈希表实现，它们在多线程环境下的行为和性能存在显著差异。以下是两者的具体分析：

1. 线程安全性

- **HashMap**：不是线程安全的，当多个线程同时对HashMap进行修改时可能会导致不可见的结果[^1^]。
- **ConcurrentHashMap**：是线程安全的，可以在多线程环境中使用而不会出现数据不一致的问题[^1^]。

2. 锁机制

- **HashMap**：没有内置锁机制，适用于单线程环境或只读多线程环境[^1^]。
- **ConcurrentHashMap**：采用分段锁设计，每个段可以独立加锁，提高了并发性能[^1^][^2^]。

3. 内部结构

- **HashMap**：JDK1.8之前是数组+链表结构，JDK1.8之后引入了红黑树来提高性能（链表长度超过阈值时转换为红黑树）[^2^]。
- **ConcurrentHashMap**：JDK1.8之前是segment数组+数组+链表，JDK1.8之后结构与HashMap类似，但增加了线程安全的特性[^2^]。

4. 性能特点

- **HashMap**：由于不需要考虑同步操作，通常在单线程应用中具有较好的性能[^1^]。
- **ConcurrentHashMap**：适用于多线程环境下频繁的读写操作，特别是写操作较多时，能提供更好的并发性能[^1^]。

综上所述，HashMap适用于单线程或只读多线程场景，因为它提供了较高的性能；而ConcurrentHashMap则专为多线程并发读写操作设计，通过分段锁等机制确保线程安全，并在高并发情况下表现出色。选择哪种取决于具体应用场景，特别是在涉及多线程时，ConcurrentHashMap通常是更合适的选择。

13. ConcurrentHashMap 和 Hashtable 的区别

ConcurrentHashMap 和 Hashtable 的区别主要体现在实现线程安全的方式上不同。

底层数据结构：JDK1.7 的 `ConcurrentHashMap` 底层采用 **分段的数组+链表** 实现，JDK1.8 采用的数据结构跟 `HashMap1.8` 的结构一样，数组+链表/红黑二叉树。`Hashtable` 和 JDK1.8 之前的 `HashMap` 的底层数据结构类似都是采用 **数组+链表** 的形式，数组是 `HashMap` 的主体，链表则是主要为了解决哈希冲突而存在的；

实现线程安全的方式（重要）：

- 在 JDK1.7 的时候，`ConcurrentHashMap` 对整个桶数组进行了分割分段(`Segment`，分段锁)，每一把锁只锁容器其中一部分数据（下面有示意图），多线程访问容器里不同数据段的数据，就不会存在锁竞争，提高并发访问率。
- 到了 JDK1.8 的时候，`ConcurrentHashMap` 已经摒弃了 `Segment` 的概念，而是直接用 `Node` 数组+链表+红黑树的数据结构来实现，并发控制使用 `synchronized` 和 CAS 来操作。（JDK1.6 以后 `synchronized` 锁做了很多优化）整个看起来就像是优化过且线程安全的 `HashMap`，虽然在 JDK1.8 中还能看到 `Segment` 的数据结构，但是已经简化了属性，只是为了兼容旧版本；
- **Hashtable (同一把锁)**:使用 `synchronized` 来保证线程安全，效率非常低下。当一个线程访问同步方法时，其他线程也访问同步方法，可能会进入阻塞或轮询状态，如使用 `put` 添加元素，另一个线程不能使用 `put` 添加元素，也不能使用 `get`，竞争会越来越激烈效率越低。

14.ConcurrentHashMap 线程安全的具体实现方式/底层具体实现

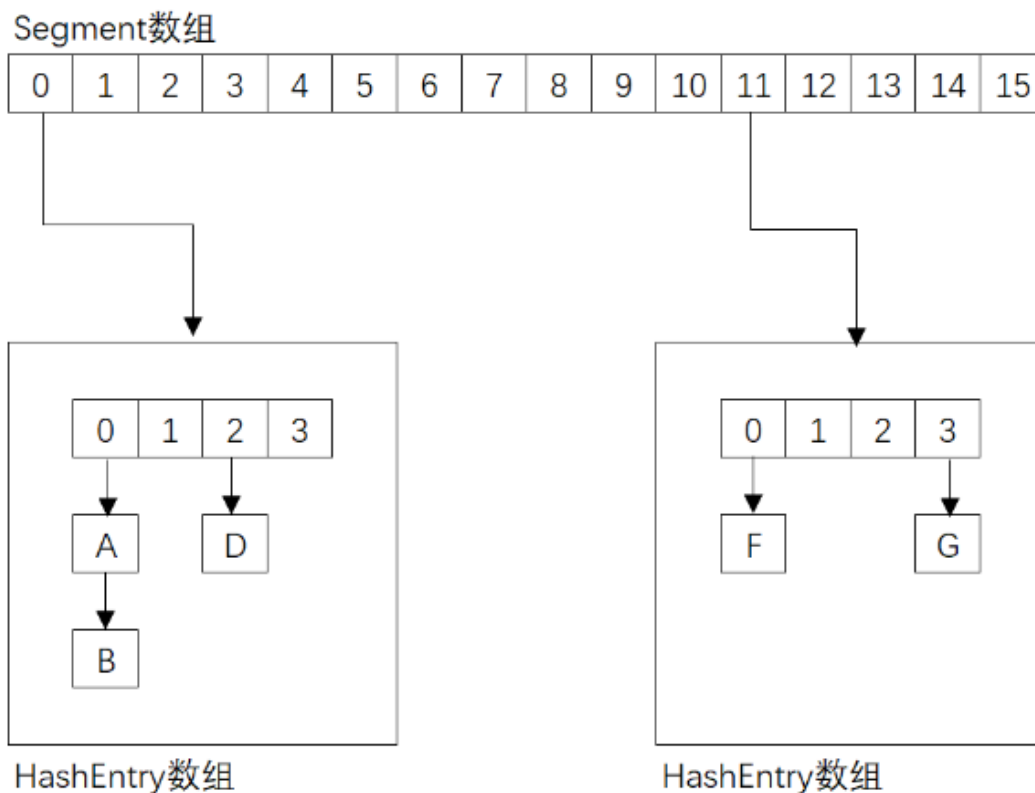
jdk 1.7

`ConcurrentHashMap` 是由 `Segment` 数组结构和+ `HashEntry` 数组结构组成。

jdk 1.7 中，`ConcurrentHashMap` 是由 `Segment` 数据结构和 `HashEntry` 数组结构构成，采取分段锁（`Segment lock`,继承自`ReentrantLock`）来保证安全性。一个 `ConcurrentHashMap` 里包含一个 `Segment` 数组，一个 `Segment` 里包含一个 `HashEntry` 数组，`HashEntry` 数组的结构和 `HashMap` 类似，是一个数组和链表结构。

首先将数据分为一段一段（这个“段”就是 `Segment`）的存储，然后给每一段数据配一把锁，当一个线程占用锁访问其中一个段数据时，其他段的数据也能被其他线程访问，就可以提高并发性。`Segment` 继承了 `ReentrantLock` ,扮演锁的角色。`HashEntry` 用于存储键值对数据。

当对 `HashEntry` 数组的数据进行修改时，必须首先获得对应的 `Segment` 的锁。也就是说，对同一 `Segment` 的并发写入会被阻塞，不同 `Segment` 的写入是可以并发执行的。



jdk 1.8

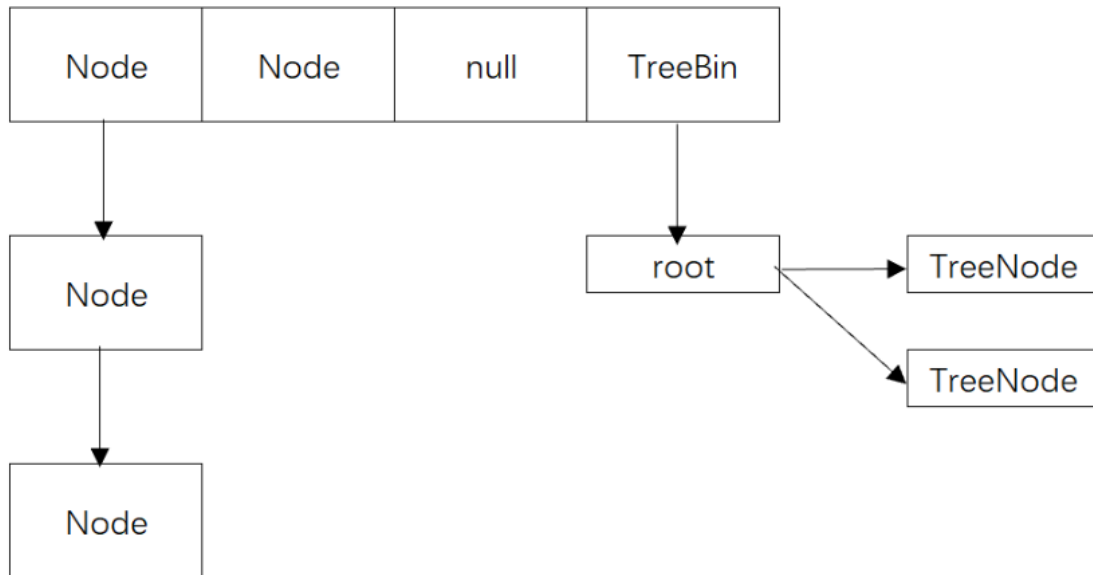
JDK1.8 的实现已经摒弃了 `Segment` 的概念，而是直接用 `Node` 数组+链表+红黑树的数据结构来实现，

并发控制使用 `Synchronized` 和 `CAS` 来操作，Java 8 在链表长度超过一定阈值（8）时将链表（寻址时间复杂度为 $O(N)$ ）转换为红黑树（寻址时间复杂度为 $O(\log(N))$ ）。整个看起来就像是优化过且线程安全的 `HashMap`

JDK1.8 放弃了 `Segment` 分段锁的设计，采用 `CAS + synchronized` 保证线程安全，锁粒度更细，`synchronized` 只锁定当前链表或红黑二叉树的首节点。

它摒弃了原有的`Segment`分段锁，而是采用了一种粒度更细的锁机制来实现线程安全。

`ConcurrentHashMap`内部维护了一个`Node`数组。`Node`对象包含了键值对以及指向下一个`Node`的指针，用于解决hash冲突问题。在JDK 1.8版本的`ConcurrentHashMap`中，每个`Node`对象都有一个`lock`字节用于存储锁状态。当对`ConcurrentHashMap`进行`put`、`get`等操作时，会尝试获取对应`Node`的锁，如果获取成功则进行操作，操作完成后再释放锁。由于不同`Node`之间的锁是独立的，因此不同线程可以同时访问不同`Node`，从而实现并发操作。



15. JDK 1.7 和 JDK 1.8 的 ConcurrentHashMap 实现有什么不同?

线程安全实现方式: JDK 1.7 采用 `Segment` 分段锁来保证安全, `Segment` 是继承自 `ReentrantLock`。JDK1.8 放弃了 `Segment` 分段锁的设计, 采用 `CAS + synchronized` 保证线程安全, 锁粒度更细, `synchronized` 只锁定当前链表或红黑二叉树的首节点。

Hash 碰撞解决方法: JDK 1.7 采用拉链法, JDK1.8 采用拉链法结合红黑树 (链表长度超过一定阈值时, 将链表转换为红黑树)。

16. ConcurrentHashMap是怎么分段分组的?

- put操作:
当执行put操作时, 会经历两个步骤:
 - 1.判断是否需要扩容;
 - 2.定位到添加元素的位置, 将其放入 `HashEntry` 数组中。插入过程会进行第一次 `key` 的 `hash` 来定位 `Segment` 的位置, 如果该 `Segment` 还没有初始化, 即通过 `CAS` 操作进行赋值, 然后进行第二次 `hash` 操作, 找到相应的 `HashEntry` 的位置, 这里会利用继承过来的锁的特性, 在将数据插入指定的 `HashEntry` 位置时 (尾插法), 会通过继承 `ReentrantLock` 的 `tryLock()` 方法尝试去获取锁, 如果获取成功就直接插入相应的位置, 如果已经有线程获取该 `Segment` 的锁, 那当前线程会以自旋的方式去继续的调用 `tryLock()` 方法去获取锁, 超过指定次数就挂起, 等待唤醒。

16. ConcurrentHashMap 为什么 key 和 value 不能为 null?

根据Java 8的ConcurrentHashMap实现, 不允许使用null作为键或值。这样做的原因主要有以下几点:

1. **并发问题:** `ConcurrentHashMap`是为高并发环境设计的, 如果允许null作为键或值, 那么在并发环境下可能会出现问題。例如, 如果一个线程在检查一个键是否存在, 而另一个线程正在将这个键的值设置为null, 那么第一个线程可能会错误地认为这个键不存在。
2. **区分不存在的键和值为null的键:** 在`ConcurrentHashMap`中, `get`方法返回null表示映射中不存在指定的键。如果允许值为null, 那么就无法区分映射中不存在这个键, 还是这个键的值就是null。

HashMap 为什么 key 和 value 能为 null?

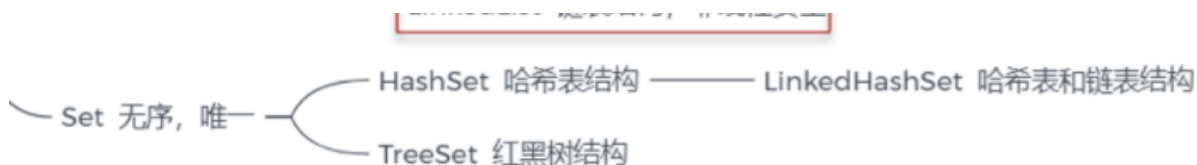
HashMap可以存储null作为键或值。以下是关于HashMap存储null的详细说明：

- **存储null为键**：HashMap允许将null作为一个键存储，但需要注意的是，HashMap中只能有一个null键。这是因为HashMap使用hash算法来存储和检索键值对，如果有两个null键，它们将具有相同的hash值（即0），这会导致键的冲突。
- **存储null为值**：与键不同，HashMap允许有多个值为null的条目。这意味着可以将null作为值与任何非空键关联。
- **内部处理机制**：当HashMap中的键为null时，其hash值会被计算为0。这是为了简化逻辑和减少歧义，因为在HashMap的设计初衷中，它是为单线程环境优化的。
- **操作方法**：可以使用HashMap的put方法将包含null键或值的条目添加到映射中，使用get方法可以通过键来检索值，即使这些键是null。遍历HashMap时，可以通过keySet()方法获取所有的键，然后逐一检索对应的值。

总的来说，HashMap在设计上允许存储null键和值，但要注意null键的唯一性。在实际应用中，应谨慎使用null键，以避免潜在的混淆和错误。

4.Set

0.Set有哪些实现类？



1.HashSet底层原理？

HashSet 基于 HashMap 实现。放入HashSet中的元素实际上由HashMap的key来保存，而HashMap的value则存储了一个静态的Object对象。

2.HashSet、LinkedHashSet 和 TreeSet 的异同？

- HashSet、LinkedHashSet 和 TreeSet 都是 Set 接口的实现类，都能保证元素唯一，并且都不是线程安全的。
- HashSet、LinkedHashSet 和 TreeSet 的主要区别在于底层数据结构不同。HashSet 的底层数据结构是哈希表（基于 HashMap 实现）。LinkedHashSet 的底层数据结构是链表和哈希表，元素的插入和取出顺序满足 FIFO。TreeSet 底层数据结构是红黑树，元素是有序的，排序的方式有自然排序和定制排序。
- 底层数据结构不同又导致这三者的应用场景不同。HashSet 用于不需要保证元素插入和取出顺序的场景，LinkedHashSet 用于保证元素的插入和取出顺序满足 FIFO 的场景，TreeSet 用于支持对元素自定义排序规则的场景。

3.HashMap 和 HashSet 区别

如果你看过 HashSet 源码的话就应该知道：HashSet 底层就是基于 HashMap 实现的。（HashSet 的源码非常非常少，因为除了 clone()、writeObject()、readObject() 是 HashSet 自己不得不实现之外，其他方法都是直接调用 HashMap 中的方法。

HashMap	HashSet
实现了 Map 接口	实现 Set 接口
存储键值对	仅存储对象
调用 put() 向 map 中添加元素	调用 add() 方法向 Set 中添加元素
HashMap 使用键 (Key) 计算 hashCode	HashSet 使用成员对象来计算 hashCode 值, 对于两个对象来说 hashCode 可能相同, 所以 equals() 方法用来判断对象的相等性

4. 说一说TreeSet和HashSet的区别

HashSet、TreeSet中的元素都是不能重复的, 并且它们都是线程不安全的, 二者的区别是:

- HashSet中的元素可以是null, 但TreeSet中的元素不能是null;
- HashSet不能保证元素的排列顺序, 而TreeSet支持自然排序、定制排序两种排序的方式;
- HashSet底层是采用哈希表实现的, 而TreeSet底层是采用红黑树实现的。

5.Queue

1.Queue 与 Deque 的区别

Queue 是单端队列, 只能从一端插入元素, 另一端删除元素, 实现上一般遵循 **先进先出 (FIFO)** 规则。

Queue 扩展了 Collection 的接口, 根据 **因为容量问题而导致操作失败后处理方式的不同** 可以分为两类方法: 一种在操作失败后会抛出异常, 另一种则会返回特殊值

Deque 是双端队列, 在队列的两端均可以插入或删除元素。

Deque 扩展了 Queue 的接口, 增加了在队首和队尾进行插入和删除的方法, 同样根据失败后处理方式的不同分为两类:

2.ArrayDeque 与 LinkedList 的区别

ArrayDeque 和 LinkedList 都实现了 Deque 接口, 两者都具有队列的功能, 但两者有什么区别呢?

- ArrayDeque 是基于可变长的数组和双指针来实现, 而 LinkedList 则通过链表来实现。
- ArrayDeque 不支持存储 NULL 数据, 但 LinkedList 支持。
- ArrayDeque 是在 JDK1.6 才被引入的, 而 LinkedList 早在 JDK1.2 时就已经存在。
- ArrayDeque 插入时可能存在扩容过程, 不过均摊后的插入操作依然为 O(1)。虽然 LinkedList 不需要扩容, 但是每次插入数据时均需要申请新的堆空间, 均摊性能相比更慢。

从性能的角度上, 选用 ArrayDeque 来实现队列要比 LinkedList 更好。此外, ArrayDeque 也可以用于实现栈

3.PriorityQueue

PriorityQueue 是在 JDK1.5 中被引入的, 其与 Queue 的区别在于元素出队顺序是与优先级相关的, 即总是优先级最高的元素先出队。

这里列举其相关的一些要点:

- `PriorityQueue` 利用了二叉堆的数据结构来实现的，底层使用可变长的数组来存储数据
- `PriorityQueue` 通过堆元素的上浮和下沉，实现了在 $O(\log n)$ 的时间复杂度内插入元素和删除堆顶元素。
- `PriorityQueue` 是非线程安全的，且不支持存储 `NULL` 和 `non-comparable` 的对象。
- `PriorityQueue` 默认是小顶堆，但可以接收一个 `Comparator` 作为构造参数，从而来自定义元素优先级的先后。

4.什么是 BlockingQueue?

`BlockingQueue` 是 `java.util.concurrent` 包下的一种重要数据结构，它提供了**线程安全**的队列访问方式。当队列为空时，取数据的线程会阻塞等待直到队列非空；当队列已满时，插入数据的线程会阻塞等待直到队列非满。这种特性使得 `BlockingQueue` 非常适合用于**生产者消费者模式**，在多线程环境中协调生产与消费速率不一致的问题。

`BlockingQueue` 是一个接口，它有几个具体的实现类，如 `ArrayBlockingQueue`、`LinkedBlockingQueue`、`PriorityBlockingQueue` 等。这些实现类在不同使用场景中各有优势。例如，`ArrayBlockingQueue` 是基于一个固定长度的数组实现的有界队列，而 `LinkedBlockingQueue` 则是基于链表实现的，其容量可以是 `Integer.MAX_VALUE`，即无界队列。

BlockingQueue是怎么实现的?

`BlockingQueue` 是通过 `ReentrantLock` 锁和条件（`Condition`）来实现的。

`BlockingQueue` 的工作原理主要依赖于 `ReentrantLock` 和 `Condition`。其中，`ReentrantLock` 是 `java.util.concurrent.locks` 包下的一个类，它实现了 `Lock` 接口并允许以排他模式持有和释放锁。**`Condition` 的 `await()` / `signal()` 机制则是用来代替传统的 `Object` 中的 `wait()` / `notify()` 机制**，可以更加灵活地控制线程之间的交互。

`BlockingDeque` 使用 `ReentrantLock` 来实现显式锁定。当一个线程尝试向已满的队列中添加元素时，它首先需要获取锁。如果成功获取锁，则可以进一步操作；否则，线程将被阻塞。当线程无法向已满的队列中添加元素时，它将被加入到 `Condition` 维护的等待队列中。这个队列是通过一个内部类 `Node` 实现的单向链表。当线程被加入等待队列后，调用 `await()` 方法，该线程将释放锁并进入等待状态。当其他线程从队列中移除元素，使空间可用时，会调用 `signal()` 或 `signalAll()` 方法唤醒等待队列中的一个或所有线程。

从使用者的角度看，`BlockingQueue` 提供四种不同的行为方式：抛异常、返回特殊值、阻塞以及超时。当操作无法立即执行时，比如向满队列中添加元素或从空队列中获取元素，这些操作会根据具体情况选择不同的行为。例如，`add(E e)` 方法在队列满时抛出 `IllegalStateException` 异常；而 `offer(E e)` 方法则在无法立即执行时返回 `false`。

综上所述，`BlockingQueue` 通过锁和条件机制解决了多线程环境下数据安全传递问题，其不同的实现类根据具体需求提供了多种选择，广泛应用于生产者消费者等并发模式中

什么是 Condition 的 await() 方法

在 `Condition` 中使用 `await()` 方法时，当前线程会释放锁并进入等待状态，直到其他线程调用 `Condition` 的 `signal()` 或者 `signalAll()` 方法唤醒它。

具体来说，当一个线程调用 `await()` 方法时，它首先会将自己放入条件队列中，并释放持有的锁。此时，该线程被阻塞，不会消耗 CPU 资源。当其他线程调用 `Condition` 的 `signal` 或 `signalAll` 方法时，等待在条件队列上的线程将被唤醒，重新尝试获取锁，并在获取成功后从 `await()` 方法返回，继续执行后续操作。

综上所述，`Condition` 的 `await()` 方法通过将线程加入条件队列并释放锁，实现对共享资源访问的精确控制，从而解决线程间的同步问题。

1. `await()`

`await()` 方法是在 `java.util.concurrent.locks.Condition` 接口中定义的，它要求一个锁（`Lock` 实例）和一个与之关联的条件变量。当一个线程调用某个条件变量的 `await()` 方法时，它会释放这个锁，并将自己阻塞，直到其他线程调用了相应的条件变量的 `signal()` 或 `signalAll()` 方法唤醒它。`await()` 方法会一直等待，直到满足以下条件之一：

- 另一个线程调用了相同的 `Condition` 对象上的 `signal()` 或 `signalAll()` 方法。
- 等待线程被中断。

2. `signal()`

`signal()` 方法同样是在 `java.util.concurrent.locks.Condition` 接口中定义的。当一个线程持有锁并调用 `signal()` 方法时，它会从等待队列中唤醒一个线程（如果有多个线程在等待，具体唤醒哪个线程由实现决定）。被唤醒的线程将会获得锁并继续执行。调用 `signal()` 并不会立即让另一个线程开始运行；被唤醒的线程必须等待当前线程释放锁。

自定义阻塞队列(Reentrantlock+Condition)

```
import java.util.LinkedList;
import java.util.concurrent.locks.Condition;
import java.util.concurrent.locks.Lock;
import java.util.concurrent.locks.ReentrantLock;

// 定义一个泛型类，可以存储任何类型的数据
public class BlockingQueue<T> {
    private LinkedList<T> queue = new LinkedList<>(); // 内部使用链表来实现队列的功能
    private int capacity; // 队列的最大容量

    // 使用可重入锁来保证线程安全
    private Lock lock = new ReentrantLock();
    // 条件变量，当队列非满时，允许生产者继续添加元素
    private Condition notFull = lock.newCondition();
    // 条件变量，当队列非空时，允许消费者继续移除元素
    private Condition notEmpty = lock.newCondition();
    // 构造函数，传入队列的容量
    public BlockingQueue(int capacity) {
        this.capacity = capacity;
    }

    // 入队操作，如果队列已满，则线程将被阻塞，直到队列有空间
    public void enqueue(T item) throws InterruptedException {
        lock.lock(); // 加锁，防止多个线程同时修改队列
        try {
            // 如果队列已满，生产者线程等待notFull条件
            while (queue.size() == capacity) {
                notFull.await();
                // 这会释放锁并使生产者线程在notFull条件上进入等待状态，
                // 到其他线程调用notFull.signalAll()方法通知它队列不再满为止。
            }
            // 未满的时候
            // 将元素添加到队列末尾
            queue.add(item);
            // 唤醒所有等待的消费者线程
            notEmpty.signalAll();
        } finally {
            lock.unlock(); // 解锁
        }
    }
}
```

```

// 出队操作，如果队列为空，则线程将被阻塞，直到队列有元素
public T dequeue() throws InterruptedException {
    lock.lock(); // 加锁
    try {
        // 如果队列为空，消费者线程等待
        while (queue.isEmpty()) {
            notEmpty.await();
        }
        // 从队列头部移除元素并返回
        T item = queue.removeFirst();
        // 唤醒所有等待的生产者线程
        notFull.signalAll();
        // 会唤醒所有在notFull条件上等待的线程（通常是生产者线程）。
        // 其中一个或多个被唤醒的生产者线程将重新获取锁，然后再次检查队列是否还有空间。
        // 如果有空间，它将添加元素并继续执行；如果没有，它会再次等待。
        return item;
    } finally {
        lock.unlock(); // 解锁
    }
}

// 测试代码
public static void main(String[] args) {
    BlockingQueue<Integer> blockingQueue = new BlockingQueue<>(5); // 创建容量
    为5的阻塞队列

    // 创建并启动生产者线程
    Thread producerThread = new Thread(() -> {
        for (int i = 0; i < 10; i++) {
            try {
                blockingQueue.enqueue(i); // 生产者入队操作
                System.out.println("生产者入队: " + i);
                //Thread.sleep(500);
            } catch (InterruptedException e) {
                e.printStackTrace();
            }
        }
    });

    // 创建并启动消费者线程
    Thread consumerThread = new Thread(() -> {
        for (int i = 0; i < 10; i++) {
            try {
                int item = blockingQueue.dequeue(); // 消费者出队操作
                System.out.println("消费者出队: " + item);
                //Thread.sleep(500);
            } catch (InterruptedException e) {
                e.printStackTrace();
            }
        }
    });
    producerThread.start();
    consumerThread.start();
}
}

```

5.ArrayBlockingQueue 和 LinkedBlockingQueue 有什么区别？

`ArrayBlockingQueue` 和 `LinkedBlockingQueue` 是 Java 并发包中常用的两种阻塞队列实现，它们都是线程安全的。不过，它们之间也存在下面这些区别：

- 底层实现：`ArrayBlockingQueue` 基于数组实现，而 `LinkedBlockingQueue` 基于链表实现。
- 是否有界：`ArrayBlockingQueue` 是有界队列，必须在创建时指定容量大小。
`LinkedBlockingQueue` 创建时可以不指定容量大小，默认是 `Integer.MAX_VALUE`，也就是无界的。但也可以指定队列大小，从而成为有界的。
- 锁是否分离：`ArrayBlockingQueue` 中的锁是没有分离的，即生产和消费用的是同一个锁；
`LinkedBlockingQueue` 中的锁是分离的，即生产用的是 `putLock`，消费是 `takeLock`，这样可以防止生产者和消费者线程之间的锁争夺。
- 内存占用：`ArrayBlockingQueue` 需要提前分配数组内存，而 `LinkedBlockingQueue` 则是动态分配链表节点内存。这意味着，`ArrayBlockingQueue` 在创建时就会占用一定的内存空间，且往往申请的内存比实际所用的内存更大，而 `LinkedBlockingQueue` 则是根据元素的增加而逐渐占用内存空间。